

LAPORAN PRAKTIKUM

Nama : Nessa Indriyanti

Nim : 25SA21A127

```
package com.mycompany.traine;  
  
import java.util.InputMismatchException;  
import java.util.Scanner;
```

Baris `package com.mycompany.traine;` digunakan untuk menentukan package tempat kelas Java berada agar sesuai dengan struktur folder pada project. Selanjutnya, `import java.util.InputMismatchException;` digunakan untuk menangani kesalahan input ketika pengguna memasukkan data yang tidak sesuai dengan tipe data yang diharapkan, sedangkan `import java.util.Scanner;` digunakan untuk menerima input dari pengguna melalui keyboard.

1. Class `traine` dan Method `stackStorage()`

```
public class Traine {  
  
    private static int[] stack = new int[5];  
    private static int counters = 0;  
  
    // Mengecek apakah stack masih memiliki ruang  
    private static boolean stackStorage() {  
        if (counters < stack.length) {  
            return true;  
        } else {  
            return false;  
        }  
    }  
}
```

Variabel `stack` digunakan sebagai media penyimpanan data stack dengan kapasitas sebanyak 5 elemen, sedangkan variabel `counters` berfungsi sebagai penunjuk jumlah data yang tersimpan di dalam stack. Method `stackStorage()` digunakan untuk mengecek apakah stack masih memiliki ruang kosong dengan membandingkan nilai `counters` dan panjang array `stack`. Jika `counters` lebih kecil dari kapasitas `stack`, maka method akan mengembalikan nilai `true`, dan sebaliknya mengembalikan nilai `false`.

2. Method createStack()

```
private static void createStack() {
    int loopX = 0;
    int alpha = 0;

    while (loopX == 0) {
        System.out.print("Masukkan Data (angka): ");
        Scanner alphaX = new Scanner(System.in);
        try {
            alpha = alphaX.nextInt();
            loopX = 1;
        } catch (InputMismatchException e) {
            System.out.println("Masukan harus berupa Angka!");
            loopX = 0;
        }
    }

    stack[counters] = alpha;
    counters++;
}
```

Method createStack() berfungsi untuk menambahkan data ke dalam stack dengan validasi input angka menggunakan try-catch. Data yang valid disimpan ke dalam stack dan jumlah data diperbarui melalui variabel counters

3. Method removeStack

```
// pop data dari stack
private static void removeStack() {
    if (counters > 0) {
        counters--;
        System.out.println("Data terakhir dalam stack sudah dikeluarkan");
    } else {
        System.out.println("Stack masih kosong!");
    }
}

// Menampilkan isi stack
```

Method removeStack() digunakan untuk menghapus satu data terakhir dari stack (pop). Program terlebih dahulu mengecek apakah stack masih berisi data dengan melihat nilai counters. Jika counters lebih besar dari nol, maka nilai counters dikurangi satu sehingga data teratas pada stack dianggap telah dikeluarkan. Sebaliknya, jika stack masih kosong, program akan menampilkan pesan bahwa stack masih kosong.

4. Method displayDataStack

```
// Menampilkan isi stack
private static void displayDataStack() {
    System.out.print("Data dalam Stack: ");
    for (int i = 0; i < counters; i++) {
        System.out.print("[ " + i + " => " + stack[i] + " ] ");
    }
    System.out.println();
}
```

Method displayDataStack() digunakan untuk menampilkan seluruh data yang tersimpan di dalam stack. Data ditampilkan menggunakan perulangan for dari indeks 0 hingga jumlah data yang

ditunjukkan oleh variabel counters. Setiap elemen stack ditampilkan beserta indeksnya, sehingga pengguna dapat melihat isi stack secara berurutan.

5. Method cleanStack dan Quitstack

```
// Membersihkan stack
private static void cleanStack() {
    counters = 0;
    System.out.println("Stack berhasil dibersihkan");
}

// Keluar dari program
private static void quitApp() {
    String quitss = "Y";
    System.out.print("Keluar dari Program? (Y/T): ");
    quitss = new Scanner(System.in).nextLine();

    if (quitss.equalsIgnoreCase("Y")) {
        System.exit(0);
    } else {
        menuProgram();
    }
}
```

Method cleanStack() digunakan untuk mengosongkan stack dengan cara mengatur nilai variabel counters menjadi 0, sehingga seluruh data yang tersimpan dianggap terhapus. Setelah itu, program menampilkan pesan bahwa stack berhasil dibersihkan. Selanjutnya, method quitApp() digunakan untuk keluar dari program dengan menampilkan konfirmasi kepada pengguna. Jika pengguna memilih "Y", maka program akan dihentikan menggunakan System.exit(0), sedangkan jika memilih selain "Y", program akan kembali ke menu utama.

6. Method MenuChooser

```
// Pemilihan menu
private static void menuChooser(int chooseMenu) {
    switch (chooseMenu) {
        case 1:
            boolean check = stackStorage();
            if (check) {
                createStack();
            } else {
                System.out.println("Stack Penuh!, kosongkan satu data terlebih dahulu!");
            }
            break;

        case 2:
            removeStack();
            break;

        case 3:
            System.out.println("Status Storage:");
            System.out.println("Kapasitas : " + stack.length);
            System.out.println("Terisi : " + counters);
            break;

        case 4:
            displayDataStack();
            break;

        case 5:
            cleanStack();
            break;
    }
}
```

Method menuChooser() berfungsi untuk mengatur dan menjalankan aksi program sesuai dengan menu yang dipilih oleh pengguna menggunakan struktur switch-case.

7. Method Menu Program

```
// Menu utama
private static void menuProgram() {
    int loopX = 0;
    int chooseMenu = 0;

    while (loopX == 0) {
        System.out.println("\nContoh Program Stack dengan Java");
        System.out.println("Menu:");
        System.out.println("1. Tambah Stack");
        System.out.println("2. Keluarkan 1 data dari Stack");
        System.out.println("3. Status Stack");
        System.out.println("4. Tampilkan data dalam Stack");
        System.out.println("5. Bersihkan Stack");
        System.out.println("6. Keluar dari Program");
        System.out.print("Pilihan Menu (1 - 6) >>> ");

        Scanner menuOption = new Scanner(System.in);
        try {
            chooseMenu = menuOption.nextInt();
            loopX = 1;
        } catch (InputMismatchException e) {
            System.out.println("Masukan harus Angka!");
        }
    }

    System.out.println();
    menuChooser(chooseMenu);
}
```

Method menuProgram() merupakan menu utama yang menampilkan seluruh pilihan operasi stack kepada pengguna. Program menggunakan perulangan while untuk memastikan pengguna memasukkan pilihan menu yang valid. Input menu dibaca menggunakan Scanner dan divalidasi dengan try-catch InputMismatchException untuk mencegah kesalahan input. Setelah pengguna memasukkan pilihan menu dengan benar, program akan memanggil method menuChooser() untuk menjalankan proses sesuai menu yang dipilih.

Running Praktikum

Contoh Program Stack dengan Java

Menu:

1. Tambah Stack
2. Keluarkan 1 data dari Stack
3. Status Stack
4. Tampilkan data dalam Stack
5. Bersihkan Stack
6. Keluar dari Program

Pilihan Menu (1 - 6) >>> 3

Status Storage:

Kapasitas : 5

Terisi : 0

Contoh Program Stack dengan Java

Menu:

1. Tambah Stack
2. Keluarkan 1 data dari Stack
3. Status Stack
4. Tampilkan data dalam Stack
5. Bersihkan Stack
6. Keluar dari Program

Pilihan Menu (1 - 6) >>> 4

Data dalam Stack:

Pertama, pengguna memilih menu nomor 3 untuk melakukan pengecekan status storage. Pada menu ini, program menampilkan kapasitas stack serta jumlah data yang telah dimasukkan ke dalam storage. Selanjutnya, pengguna memilih menu nomor 4 yang berfungsi untuk menampilkan data-data yang telah diinputkan ke dalam stack.

Contoh Program Stack dengan Java

Menu:

1. Tambah Stack
2. Keluarkan 1 data dari Stack
3. Status Stack
4. Tampilkan data dalam Stack
5. Bersihkan Stack
6. Keluar dari Program

Pilihan Menu (1 - 6) >>> 1

Contoh Program Stack dengan Java

Menu:

1. Tambah Stack
2. Keluarkan 1 data dari Stack
3. Status Stack
4. Tampilkan data dalam Stack
5. Bersihkan Stack
6. Keluar dari Program

Pilihan Menu (1 - 6) >>> 1

Masukkan Data (angka): 1

Setelah itu saya mencoba untuk menginputkan angka 1 untuk menambahkan stok dan saya memasukan data dengan number 1 dimana nanti data angka tersimpan pada storage .

Contoh Program Stack dengan Java

Menu:

1. Tambah Stack
 2. Keluarkan 1 data dari Stack
 3. Status Stack
 4. Tampilkan data dalam Stack
 5. Bersihkan Stack
 6. Keluar dari Program
- Pilihan Menu (1 - 6) >>> 4

Data dalam Stack: [0 => 1] [1 => 2] [2 => 3] [3 => 4] [4 => 5]

Contoh Program Stack dengan Java

Menu:

1. Tambah Stack
 2. Keluarkan 1 data dari Stack
 3. Status Stack
 4. Tampilkan data dalam Stack
 5. Bersihkan Stack
 6. Keluar dari Program
- Pilihan Menu (1 - 6) >>> 1

Stack Penuh!, kosongkan satu data terlebih dahulu!

1. Tambah Stack
 2. Keluarkan 1 data dari Stack
 3. Status Stack
 4. Tampilkan data dalam Stack
 5. Bersihkan Stack
 6. Keluar dari Program
- Pilihan Menu (1 - 6) >>> 4

Data dalam Stack: [0 => 1] [1 => 2] [2 => 3] [3 => 4]

Contoh Program Stack dengan Java

Menu:

1. Tambah Stack
 2. Keluarkan 1 data dari Stack
 3. Status Stack
 4. Tampilkan data dalam Stack
 5. Bersihkan Stack
 6. Keluar dari Program
- Pilihan Menu (1 - 6) >>> 1

Masukkan Data (angka): 6

Contoh Program Stack dengan Java

Menu:

1. Tambah Stack
 2. Keluarkan 1 data dari Stack
 3. Status Stack
 4. Tampilkan data dalam Stack
 5. Bersihkan Stack
 6. Keluar dari Program
- Pilihan Menu (1 - 6) >>> 4

Data dalam Stack: [0 => 1] [1 => 2] [2 => 3] [3 => 4] [4 => 6]

1. Tambah Stack
 2. Keluarkan 1 data dari Stack
 3. Status Stack
 4. Tampilkan data dalam Stack
 5. Bersihkan Stack
 6. Keluar dari Program
- Pilihan Menu (1 - 6) >>> 4

Data dalam Stack: [0 => 1] [1 => 2] [2 => 3] [3 => 4] [4 => 5]

Contoh Program Stack dengan Java

Menu:

1. Tambah Stack
 2. Keluarkan 1 data dari Stack
 3. Status Stack
 4. Tampilkan data dalam Stack
 5. Bersihkan Stack
 6. Keluar dari Program
- Pilihan Menu (1 - 6) >>> 2

Data terakhir dalam stack sudah dikeluarkan

Contoh Program Stack dengan Java

Menu:

1. Tambah Stack
 2. Keluarkan 1 data dari Stack
 3. Status Stack
 4. Tampilkan data dalam Stack
 5. Bersihkan Stack
 6. Keluar dari Program
- Pilihan Menu (1 - 6) >>> 4

Data dalam Stack: [0 => 1] [1 => 2] [2 => 3] [3 => 4]

Selanjutnya, pengguna mencoba memasukkan data ke dalam stack secara berulang hingga jumlah data yang tersimpan mencapai kapasitas maksimum, yaitu sebanyak 5 data. Setelah stack penuh, ketika pengguna kembali memilih menu untuk menambahkan data, program menampilkan pesan bahwa stack telah penuh dan meminta pengguna untuk mengosongkan satu data terlebih dahulu. Kemudian, pengguna memilih menu nomor 2 untuk mengeluarkan satu data dari stack. Setelah data berhasil dikeluarkan, jumlah data dalam stack berkurang. Selanjutnya, pengguna kembali menambahkan data baru dengan memasukkan angka 6, dan data tersebut berhasil disimpan ke dalam stack.